

Mini RFC: Flow Control

April 5, 2022

Note: This is a simplified and annotated summary of the QUIC-related IETF RFCs. It's part of a course project at Tsinghua University and not suitable for any other purposes.

We reorder paragraphs in QUIC RFCs and omit many details to improve its readability for beginners.

WARNING: DO NOT distribute this document. Modification of extracts from IETF RFCs is not granted by IETF as original authors retain copyrights of RFCs. This document is for educational purposes only.

1 Overview

Receivers need to limit the amount of data that they are required to buffer, in order to prevent a fast sender from overwhelming them or a malicious sender from consuming a large amount of memory. To enable a receiver to limit memory commitments for a connection, streams are flow controlled both individually and across a connection as a whole. A QUIC receiver controls the maximum amount of data the sender can send on a stream as well as across all streams at any time.

Similarly, to limit concurrency within a connection, a QUIC endpoint controls the maximum cumulative number of streams that its peer can initiate.

2 Data Flow Control

QUIC employs a limit-based flow control scheme where a receiver advertises the limit of total bytes it is prepared to receive on a given stream or for the entire connection. This leads to two levels of data flow control in QUIC:

- Stream flow control, which prevents a single stream from consuming the entire receive buffer for a connection by limiting the amount of data that can be sent on each stream.
- Connection flow control, which prevents senders from exceeding a receiver's buffer capacity for the connection by limiting the total bytes of stream data sent in STREAM frames on all streams.

Senders MUST NOT send data in excess of either limit.

A receiver sets initial limits for all streams through transport parameters during the handshake. Subsequently, a receiver sends MAX_STREAM_DATA frames or MAX_DATA frames to the sender to advertise larger limits.

A receiver can advertise a larger limit for a stream by sending a MAX_STREAM_DATA frame with the corresponding stream ID. A MAX_STREAM_DATA frame indicates the maximum absolute byte offset of a stream. A receiver could determine the flow control offset to be advertised based on the current offset of data consumed on that stream.

A receiver can advertise a larger limit for a connection by sending a MAX_DATA frame, which indicates the maximum of the sum of the absolute byte offsets of all streams. A receiver maintains a cumulative sum of bytes received on all streams, which is used to check for violations of the advertised connection or stream data limits. A receiver could determine the maximum data limit to be advertised based on the sum of bytes consumed on all streams.

Once a receiver advertises a limit for the connection or a stream, it is not an error to advertise a smaller limit, but the smaller limit has no effect.

A receiver MUST close the connection with an error of type FLOW_CONTROL_ERROR if the sender violates the advertised connection or stream data limits.

A sender MUST ignore any MAX_STREAM_DATA or MAX_DATA frames that do not increase flow control limits.

If a sender has sent data up to the limit, it will be unable to send new data and is considered blocked. A sender SHOULD send a STREAM_DATA_BLOCKED or DATA_BLOCKED frame to indicate to the receiver that it has data to write but is blocked by flow control limits. If a sender is blocked for a period longer than the idle timeout, the receiver might close the connection even when the sender has data that is available for transmission. To keep the connection from closing, a sender that is flow control limited SHOULD periodically send a STREAM_DATA_BLOCKED or DATA_BLOCKED frame when it has no ack-eliciting packets in flight.

2.1 Stream State Machine

2.1.1 Sending

Figure 1 shows the states for the part of a stream that sends data to a peer.

The sending part of a stream that the endpoint initiates (types 0 and 2 for clients, 1 and 3 for servers) is opened by the application. The “Ready” state represents a newly created stream that is able to accept data from the application. Stream data might be buffered in this state in preparation for sending.

Sending the first STREAM or STREAM_DATA_BLOCKED frame causes a sending part of a stream to enter the “Send” state. An implementation might choose to defer allocating a stream ID to a stream until it sends the first STREAM frame and enters this state, which can allow for better stream prioritization.

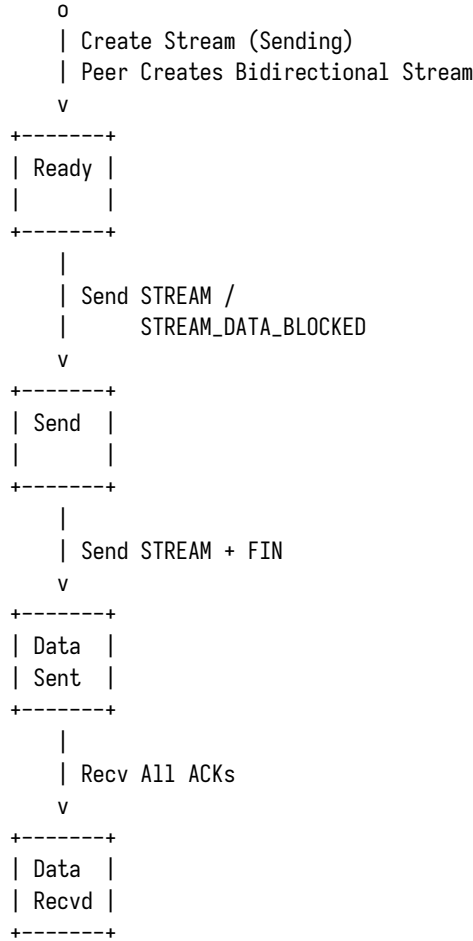


Figure 1: *States for Sending Parts of Streams*

The sending part of a bidirectional stream initiated by a peer (type 0 for a server, type 1 for a client) starts in the “Ready” state when the receiving part is created.

In the “Send” state, an endpoint transmits – and retransmits as necessary – stream data in STREAM frames. The endpoint respects the flow control limits set by its peer and continues to accept and process MAX_STREAM_DATA frames. An endpoint in the “Send” state generates STREAM_DATA_BLOCKED frames if it is blocked from sending by stream flow control limits.

After the application indicates that all stream data has been sent and a STREAM frame containing the FIN bit is sent, the sending part of the stream enters the “Data Sent” state. From this state, the endpoint only retransmits stream data as necessary. The endpoint does not need to check flow control limits or send STREAM_DATA_BLOCKED frames for a stream in this state.

MAX_STREAM_DATA frames might be received until the peer receives the final stream offset. The endpoint can safely ignore any MAX_STREAM_DATA frames it receives from its peer for a stream in this state.

Once all stream data has been successfully acknowledged, the sending part of the stream enters the “Data Recvd” state, which is a terminal state.

2.1.2 Receiving

Figure 2 shows the states for the part of a stream that receives data from a peer. The states for a receiving part of a stream mirror only some of the states of the sending part of the stream at the peer. The receiving part of a stream does not track states on the sending part that cannot be observed, such as the “Ready” state. Instead, the receiving part of a stream tracks the delivery of data to the application, some of which cannot be observed by the sender.

The receiving part of a stream initiated by a peer (types 1 and 3 for a client, or 0 and 2 for a server) is created when the first STREAM, STREAM_DATA_BLOCKED, or RESET_STREAM frame is received for that stream. For bidirectional streams initiated by a peer, receipt of a MAX_STREAM_DATA or STOP_SENDING frame for the sending part of the stream also creates the receiving part. The initial state for the receiving part of a stream is “Recv”.

Check [mini RFC: Advanced Transmission](#) for more details on stream abort and reset.

For a bidirectional stream, the receiving part enters the “Recv” state when the sending part initiated by the endpoint (type 0 for a client, type 1 for a server) enters the “Ready” state.

An endpoint opens a bidirectional stream when a MAX_STREAM_DATA or STOP_SENDING frame is received from the peer for that stream. Receiving a MAX_STREAM_DATA frame for an unopened stream indicates that the remote peer has opened the stream and is providing flow control credit. Receiving a STOP_SENDING frame for an unopened stream indicates that the remote peer no longer wishes to receive data on this stream. Either frame might arrive before a STREAM or STREAM_DATA_BLOCKED frame if packets are lost or reordered.

Before a stream is created, all streams of the same type with lower-numbered stream IDs MUST be created. This ensures that the creation order for streams is consistent on both endpoints.

In the “Recv” state, the endpoint receives STREAM and STREAM_DATA_BLOCKED frames. Incoming data is buffered and can be reassembled into the correct order for delivery to the application. As data is consumed by the application and buffer space becomes available, the endpoint sends MAX_STREAM_DATA frames to allow the peer to send more data.

When a STREAM frame with a FIN bit is received, the final size of the stream is known. The receiving part of the stream then enters the “Size Known” state. In this state, the endpoint no longer needs to send MAX_STREAM_DATA frames; it only receives any retransmissions of stream data.

Once all data for the stream has been received, the receiving part enters the “Data Recvd” state. This might happen as a result of receiving the same STREAM frame that causes the transition to “Size Known”. After all data has

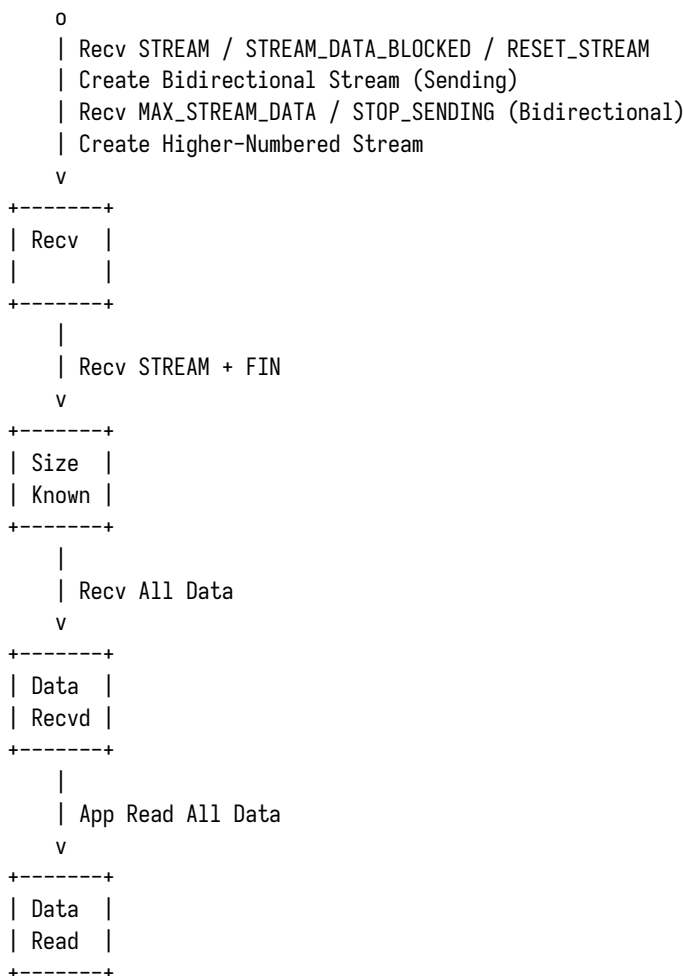


Figure 2: *States for Receiving Parts of Streams*

been received, any STREAM or STREAM_DATA_BLOCKED frames for the stream can be discarded.

The “Data Recvd” state persists until stream data has been delivered to the application. Once stream data has been delivered, the stream enters the “Data Read” state, which is a terminal state.

2.2 Increasing Flow Control Limits

Implementations decide when and how much credit to advertise in MAX_STREAM_DATA or MAX_DATA frames, but this section offers a few considerations.

To avoid blocking a sender, a receiver MAY send a MAX_STREAM_DATA or MAX_DATA frame multiple times within a round trip or send it early enough to allow time for loss of the frame and subsequent recovery.

Control frames contribute to connection overhead. Therefore, frequently sending `MAX_STREAM_DATA` and `MAX_DATA` frames with small changes is undesirable. On the other hand, if updates are less frequent, larger increments to limits are necessary to avoid blocking a sender, requiring larger resource commitments at the receiver. There is a trade-off between resource commitment and overhead when determining how large a limit is advertised.

A receiver can use an autotuning mechanism to tune the frequency and amount of advertised additional credit based on a round-trip time estimate and the rate at which the receiving application consumes data, similar to common TCP implementations. As an optimization, an endpoint could send frames related to flow control only when there are other frames to send, ensuring that flow control does not cause extra packets to be sent.

A blocked sender is not required to send `STREAM_DATA_BLOCKED` or `DATA_BLOCKED` frames. Therefore, a receiver **MUST NOT** wait for a `STREAM_DATA_BLOCKED` or `DATA_BLOCKED` frame before sending a `MAX_STREAM_DATA` or `MAX_DATA` frame; doing so could result in the sender being blocked for the rest of the connection. Even if the sender sends these frames, waiting for them will result in the sender being blocked for at least an entire round trip.

2.3 Flow Control Performance

If an endpoint cannot ensure that its peer always has available flow control credit that is greater than the peer's bandwidth-delay product on this connection, its receive throughput will be limited by flow control.

Packet loss can cause gaps in the receive buffer, preventing the application from consuming data and freeing up receive buffer space.

Sending timely updates of flow control limits can improve performance. Sending packets only to provide flow control updates can increase network load and adversely affect performance. Sending flow control updates along with other frames, such as ACK frames, reduces the cost of those updates.

2.4 Handling Stream Cancellation

Endpoints need to eventually agree on the amount of flow control credit that has been consumed on every stream, to be able to account for all bytes for connection-level flow control.

On receipt of a `RESET_STREAM` frame, an endpoint will tear down state for the matching stream and ignore further data arriving on that stream.

`RESET_STREAM` terminates one direction of a stream abruptly. For a bidirectional stream, `RESET_STREAM` has no effect on data flow in the opposite direction. Both endpoints **MUST** maintain flow control state for the stream in the unterminated direction until that direction enters a terminal state.

2.5 Stream Final Size

The final size is the amount of flow control credit that is consumed by a stream. Assuming that every contiguous byte on the stream was sent once,

the final size is the number of bytes sent. More generally, this is one higher than the offset of the byte with the largest offset sent on the stream, or zero if no bytes were sent.

A sender always communicates the final size of a stream to the receiver reliably, no matter how the stream is terminated. The final size is the sum of the Offset and Length fields of a STREAM frame with a FIN flag, noting that these fields might be implicit. Alternatively, the Final Size field of a RESET_STREAM frame carries this value. This guarantees that both endpoints agree on how much flow control credit was consumed by the sender on that stream.

An endpoint will know the final size for a stream when the receiving part of the stream enters the "Size Known" or "Reset Recvd" state. The receiver MUST use the final size of the stream to account for all bytes sent on the stream in its connection-level flow controller.

An endpoint MUST NOT send data on a stream at or beyond the final size.

Once a final size for a stream is known, it cannot change. If a RESET_STREAM or STREAM frame is received indicating a change in the final size for the stream, an endpoint SHOULD respond with an error of type FINAL_SIZE_ERROR. A receiver SHOULD treat receipt of data at or beyond the final size as an error of type FINAL_SIZE_ERROR, even after a stream is closed. Generating these errors is not mandatory, because requiring that an endpoint generate these errors also means that the endpoint needs to maintain the final size state for closed streams, which could mean a significant state commitment.

2.6 Controlling Concurrency

An endpoint limits the cumulative number of incoming streams a peer can open. Only streams with a stream ID less than $(\text{max_streams} * 4 + \text{first_stream_id_of_type})$ can be opened. Initial limits are set in the transport parameters. Subsequent limits are advertised using MAX_STREAMS frames. Separate limits apply to unidirectional and bidirectional streams.

If a max_streams transport parameter or a MAX_STREAMS frame is received with a value greater than 2^{60} , this would allow a maximum stream ID that cannot be expressed as a variable-length integer. If either is received, the connection MUST be closed immediately with a connection error of type TRANSPORT_PARAMETER_ERROR if the offending value was received in a transport parameter or of type FRAME_ENCODING_ERROR if it was received in a frame.

Endpoints MUST NOT exceed the limit set by their peer. An endpoint that receives a frame with a stream ID exceeding the limit it has sent MUST treat this as a connection error of type STREAM_LIMIT_ERROR.

Once a receiver advertises a stream limit using the MAX_STREAMS frame, advertising a smaller limit has no effect. MAX_STREAMS frames that do not increase the stream limit MUST be ignored.

As with stream and connection flow control, this document leaves implementations to decide when and how many streams should be advertised

to a peer via MAX_STREAMS. Implementations might choose to increase limits as streams are closed, to keep the number of streams available to peers roughly consistent.

An endpoint that is unable to open a new stream due to the peer's limits SHOULD send a STREAMS_BLOCKED frame. This signal is considered useful for debugging. An endpoint MUST NOT wait to receive this signal before advertising additional credit, since doing so will mean that the peer will be blocked for at least an entire round trip, and potentially indefinitely if the peer chooses not to send STREAMS_BLOCKED frames.

3 Packets and Frames

3.1 Packets

There is no new type of packets introduced in this RFC.

3.2 Frames

3.2.1 MAX_DATA Frames

A MAX_DATA frame (type=0x10) is used in flow control to inform the peer of the maximum amount of data that can be sent on the connection as a whole.

```
MAX_DATA Frame {
  Type (i) = 0x10,
  Maximum Data (i),
}
```

Figure 3: MAX_DATA Frame Format

- **Maximum Data:** A variable-length integer indicating the maximum amount of data that can be sent on the entire connection, in units of bytes.

All data sent in STREAM frames counts toward this limit. The sum of the final sizes on all streams – including streams in terminal states – MUST NOT exceed the value advertised by a receiver. An endpoint MUST terminate a connection with an error of type FLOW_CONTROL_ERROR if it receives more data than the maximum data value that it has sent.

3.2.2 MAX_STREAM_DATA Frames

A MAX_STREAM_DATA frame (type=0x11) is used in flow control to inform a peer of the maximum amount of data that can be sent on a stream.

A MAX_STREAM_DATA frame can be sent for streams in the "Recv" state. Receiving a MAX_STREAM_DATA frame for a locally initiated stream that has not

yet been created MUST be treated as a connection error of type `STREAM_STATE_ERROR`. An endpoint that receives a `MAX_STREAM_DATA` frame for a receive-only stream MUST terminate the connection with error `STREAM_STATE_ERROR`.

```
MAX_STREAM_DATA Frame {
    Type (i) = 0x11,
    Stream ID (i),
    Maximum Stream Data (i),
}
```

Figure 4: *MAX_STREAM_DATA Frame Format*

- **Stream ID:** The stream ID of the affected stream, encoded as a variable-length integer.
- **Maximum Stream Data:** A variable-length integer indicating the maximum amount of data that can be sent on the identified stream, in units of bytes.

When counting data toward this limit, an endpoint accounts for the largest received offset of data that is sent or received on the stream. Loss or reordering can mean that the largest received offset on a stream can be greater than the total size of data received on that stream. Receiving `STREAM` frames might not increase the largest received offset.

If we count bytes that are actually received, we may meet a tricky condition that receiving reordering data violates the flow control limit. The receiver may also disagree with the sender on the size of data have been sent by the latter if some packets disappear silently in the wire. Inconsistency never makes your life easier.

3.2.3 MAX_STREAMS Frames

A `MAX_STREAMS` frame (type=0x12 or 0x13) informs the peer of the cumulative number of streams of a given type it is permitted to open. A `MAX_STREAMS` frame with a type of 0x12 applies to bidirectional streams, and a `MAX_STREAMS` frame with a type of 0x13 applies to unidirectional streams.

```
MAX_STREAMS Frame {
    Type (i) = 0x12..0x13,
    Maximum Streams (i),
}
```

Figure 5: *MAX_STREAMS Frame Format*

`MAX_STREAMS` frames contain the following field:

- **Maximum Streams:** A count of the cumulative number of streams of the corresponding type that can be opened over the lifetime of the connection. This value cannot exceed 2^{60} , as it is not possible to encode stream

IDs larger than $2^{62} - 1$. Receipt of a frame that permits opening of a stream larger than this limit MUST be treated as a connection error of type `FRAME_ENCODING_ERROR`.

Loss or reordering can cause an endpoint to receive a `MAX_STREAMS` frame with a lower stream limit than was previously received. `MAX_STREAMS` frames that do not increase the stream limit MUST be ignored.

An endpoint MUST NOT open more streams than permitted by the current stream limit set by its peer. For instance, a server that receives a unidirectional stream limit of 3 is permitted to open streams 3, 7, and 11, but not stream 15. An endpoint MUST terminate a connection with an error of type `STREAM_LIMIT_ERROR` if a peer opens more streams than was permitted.

Note that these frames (and the corresponding transport parameters) do not describe the number of streams that can be opened concurrently. The limit includes streams that have been closed as well as those that are open.

3.2.4 DATA_BLOCKED Frames

A sender SHOULD send a `DATA_BLOCKED` frame (type=0x14) when it wishes to send data but is unable to do so due to connection-level flow control. `DATA_BLOCKED` frames can be used as input to tuning of flow control algorithms.

```
DATA_BLOCKED Frame {  
    Type (i) = 0x14,  
    Maximum Data (i),  
}
```

Figure 6: *DATA_BLOCKED Frame Format*

`DATA_BLOCKED` frames contain the following field:

- **Maximum Data:** A variable-length integer indicating the connection-level limit at which blocking occurred.

3.2.5 STREAM_DATA_BLOCKED Frames

A sender SHOULD send a `STREAM_DATA_BLOCKED` frame (type=0x15) when it wishes to send data but is unable to do so due to stream-level flow control. This frame is analogous to `DATA_BLOCKED`.

An endpoint that receives a `STREAM_DATA_BLOCKED` frame for a send-only stream MUST terminate the connection with error `STREAM_STATE_ERROR`.

`STREAM_DATA_BLOCKED` frames contain the following fields:

- **Stream ID:** A variable-length integer indicating the stream that is blocked due to flow control.

```
STREAM_DATA_BLOCKED Frame {  
  Type (i) = 0x15,  
  Stream ID (i),  
  Maximum Stream Data (i),  
}
```

Figure 7: *STREAM_DATA_BLOCKED Frame Format*

- **Maximum Stream Data:** A variable-length integer indicating the offset of the stream at which the blocking occurred.

3.2.6 STREAMS_BLOCKED Frames

A sender SHOULD send a STREAMS_BLOCKED frame (type=0x16 or 0x17) when it wishes to open a stream but is unable to do so due to the maximum stream limit set by its peer. A STREAMS_BLOCKED frame of type 0x16 is used to indicate reaching the bidirectional stream limit, and a STREAMS_BLOCKED frame of type 0x17 is used to indicate reaching the unidirectional stream limit.

A STREAMS_BLOCKED frame does not open the stream, but informs the peer that a new stream was needed and the stream limit prevented the creation of the stream.

```
STREAMS_BLOCKED Frame {  
  Type (i) = 0x16..0x17,  
  Maximum Streams (i),  
}
```

Figure 8: *STREAMS_BLOCKED Frame Format*

STREAMS_BLOCKED frames contain the following field:

- **Maximum Streams:** A variable-length integer indicating the maximum number of streams allowed at the time the frame was sent. This value cannot exceed 2^{60} , as it is not possible to encode stream IDs larger than $2^{62}-1$. Receipt of a frame that encodes a larger stream ID MUST be treated as a connection error of type `STREAM_LIMIT_ERROR` or `FRAME_ENCODING_ERROR`.

4 Application Protocol

There is no new application protocol defined in this mini RFC.